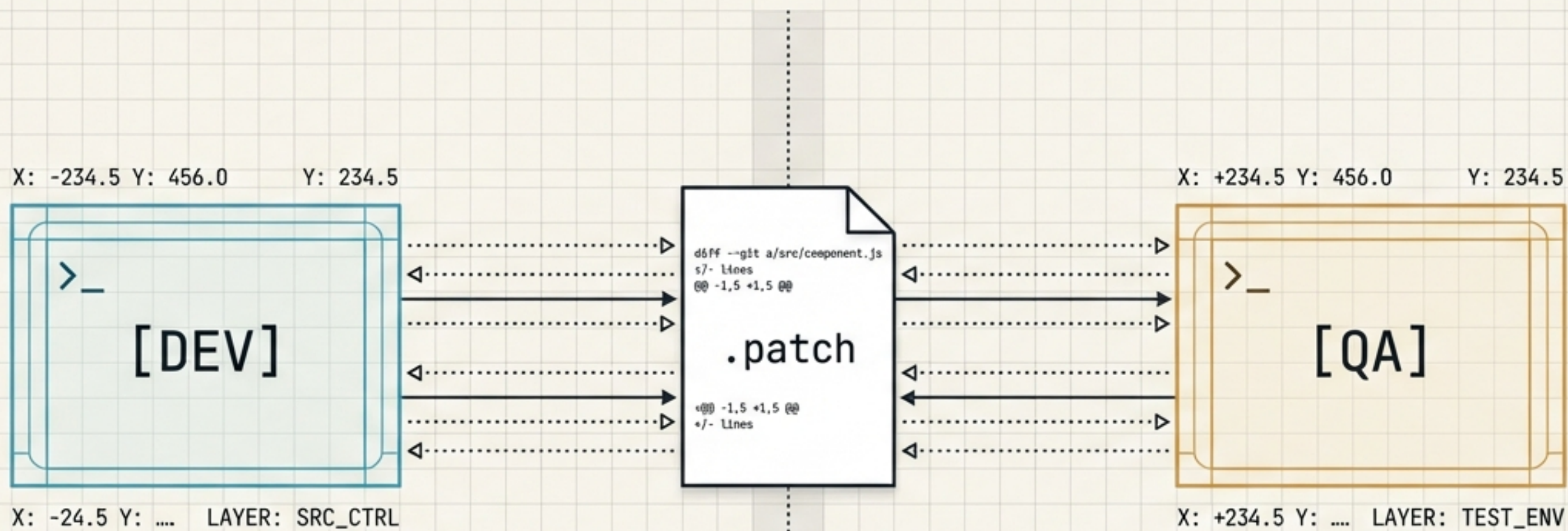
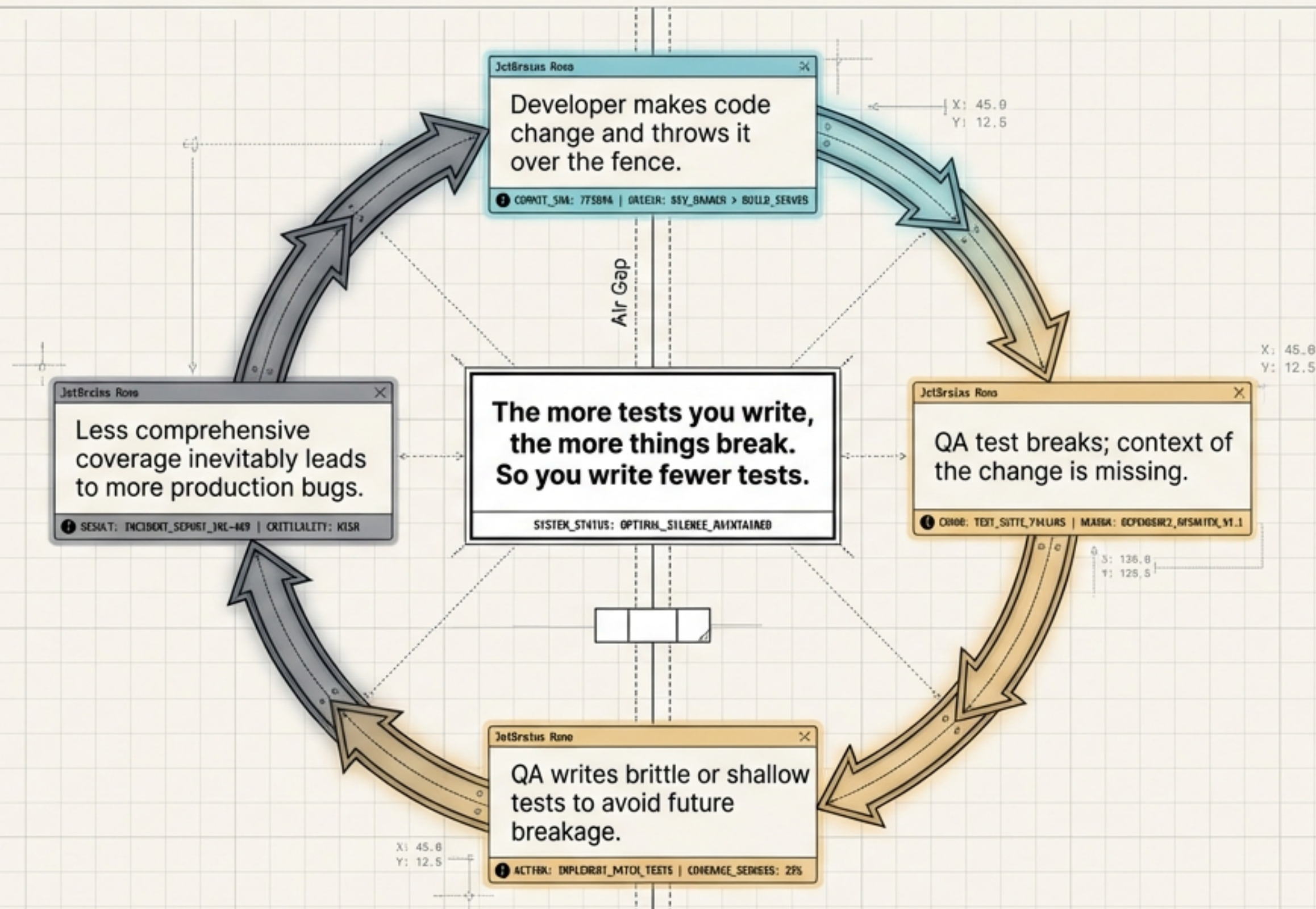


Patches as Bidirectional Communication

Redefining the Dev-to-QA Workflow for Agentic Teams

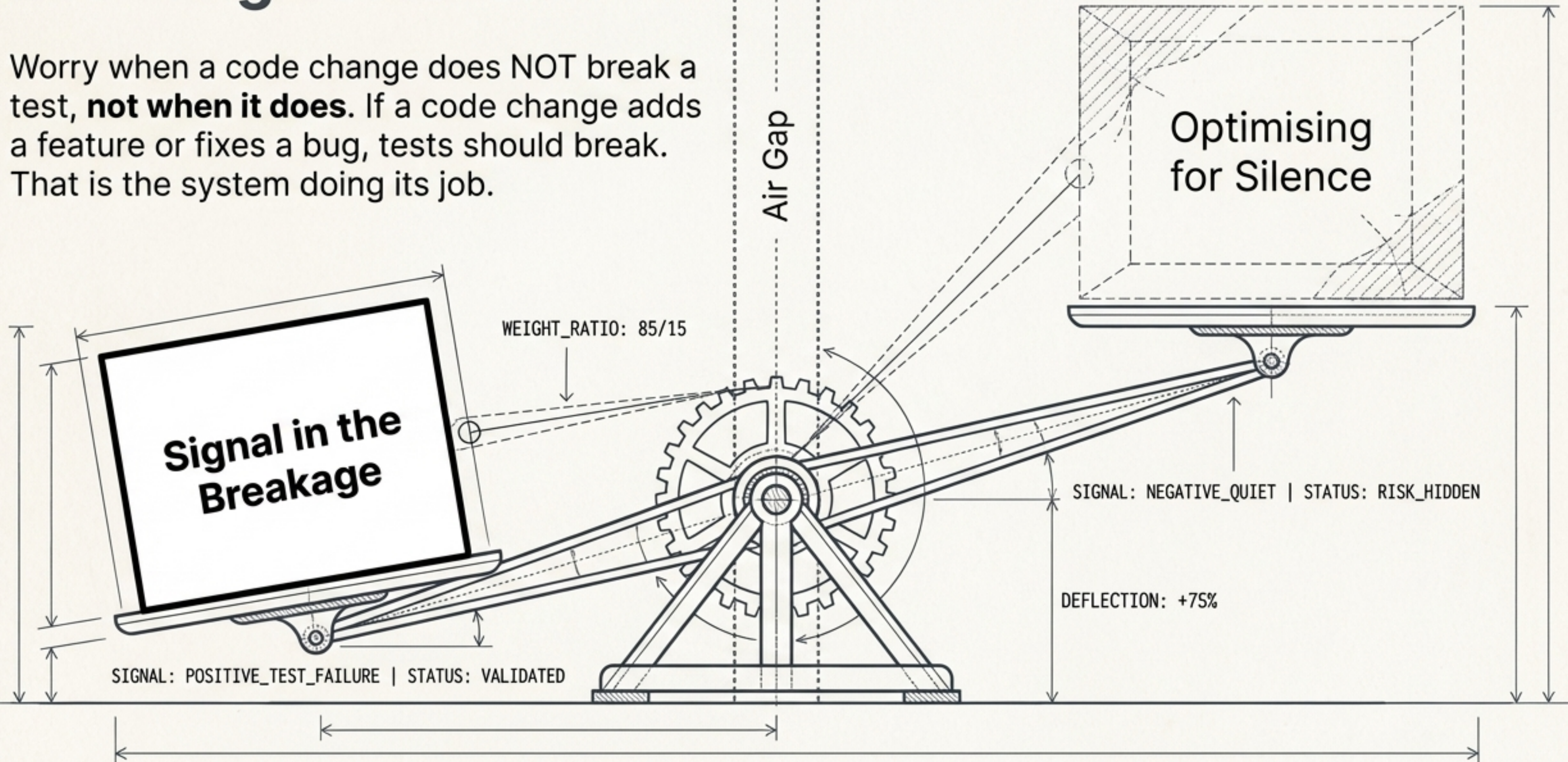


The System Optimises for Silence



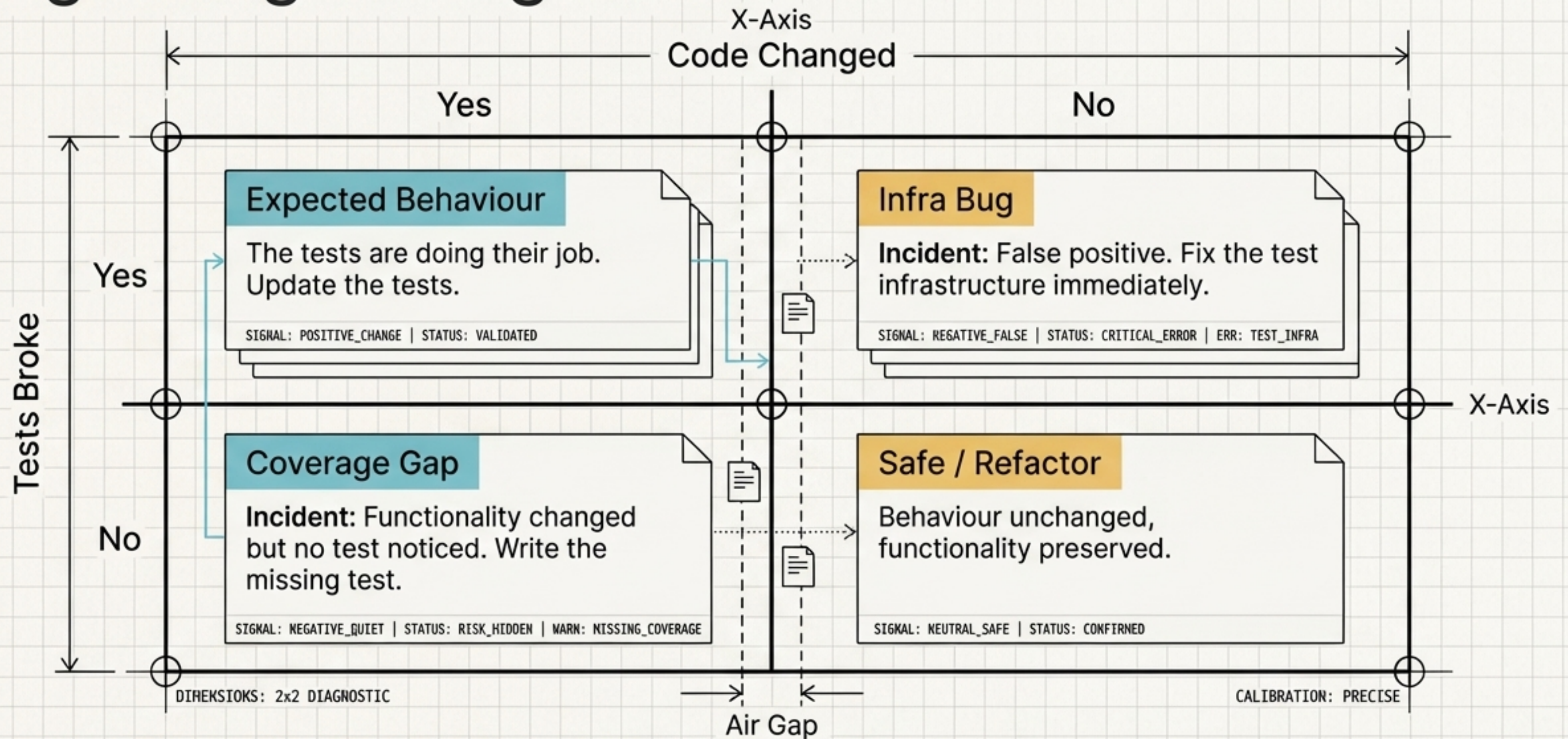
Breaking Tests is Good

Worry when a code change does NOT break a test, **not when it does**. If a code change adds a feature or fixes a bug, tests should break. That is the system doing its job.



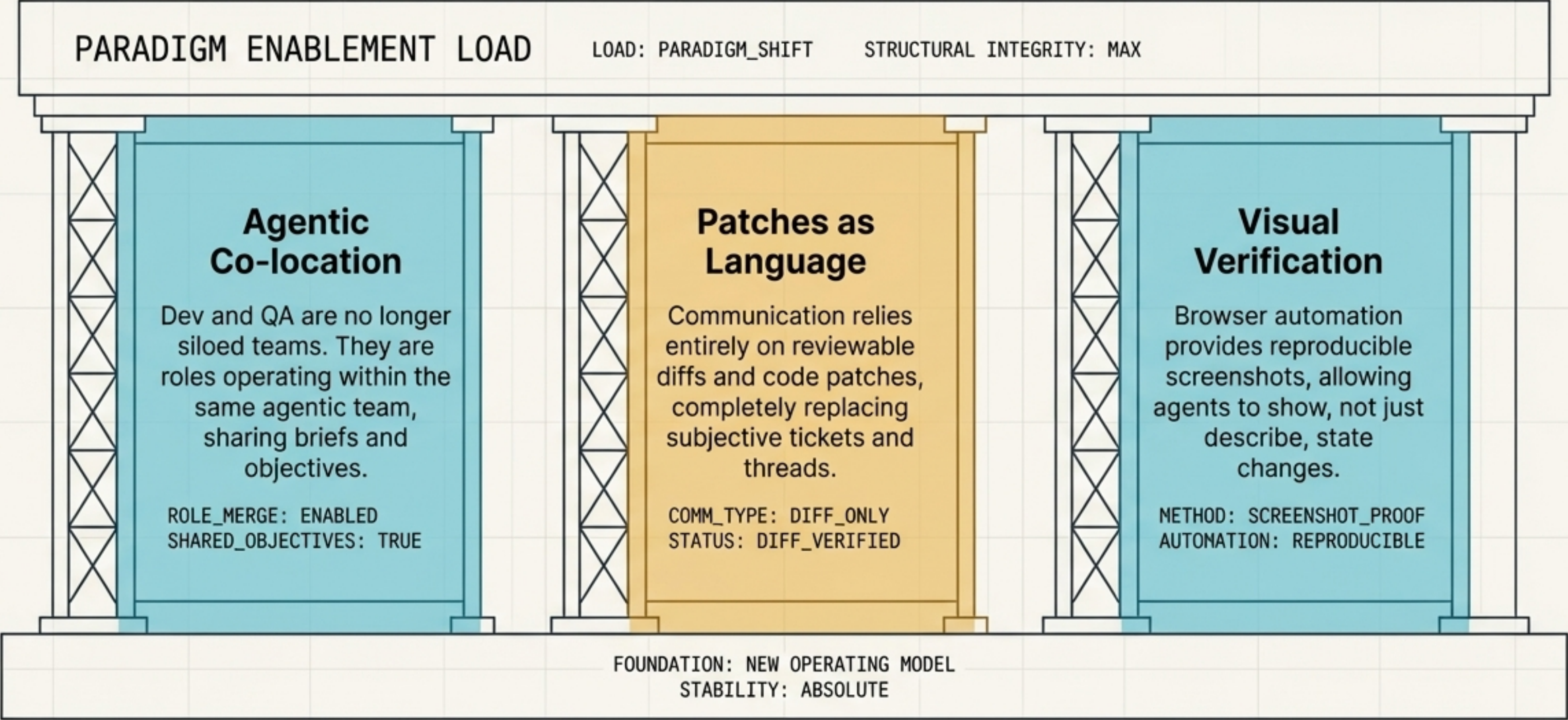
Diagnosing the Signal

DIMENSIONS: 2x2 DIAGNOSTIC
UNIT: MATRIX
CALIBRATION: PRECISE

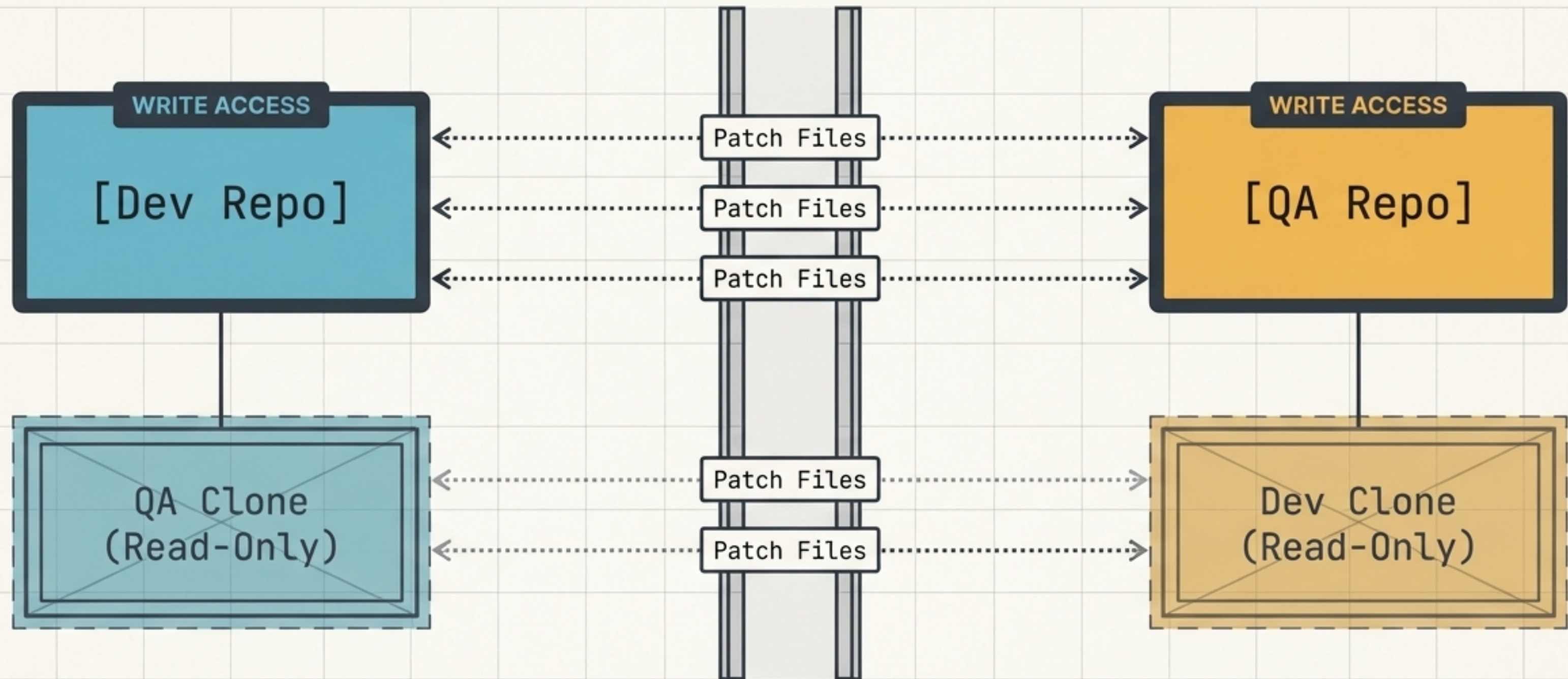


Every gap is visible. Every gap has an owner.

What Makes the New Paradigm Possible



Complete Separation. Complete Communication.



Neither side has write access to the other's repository.
Read-only clones and patch files structurally preserve the air gap.

The Developer Fixes the Breakage

Make Code Change

Developer implements the feature.

Run All Tests

Execute Unit, Integration, and E2E tests locally.

Observe Breakage

Tests fail due to the new behaviour (expected outcome).

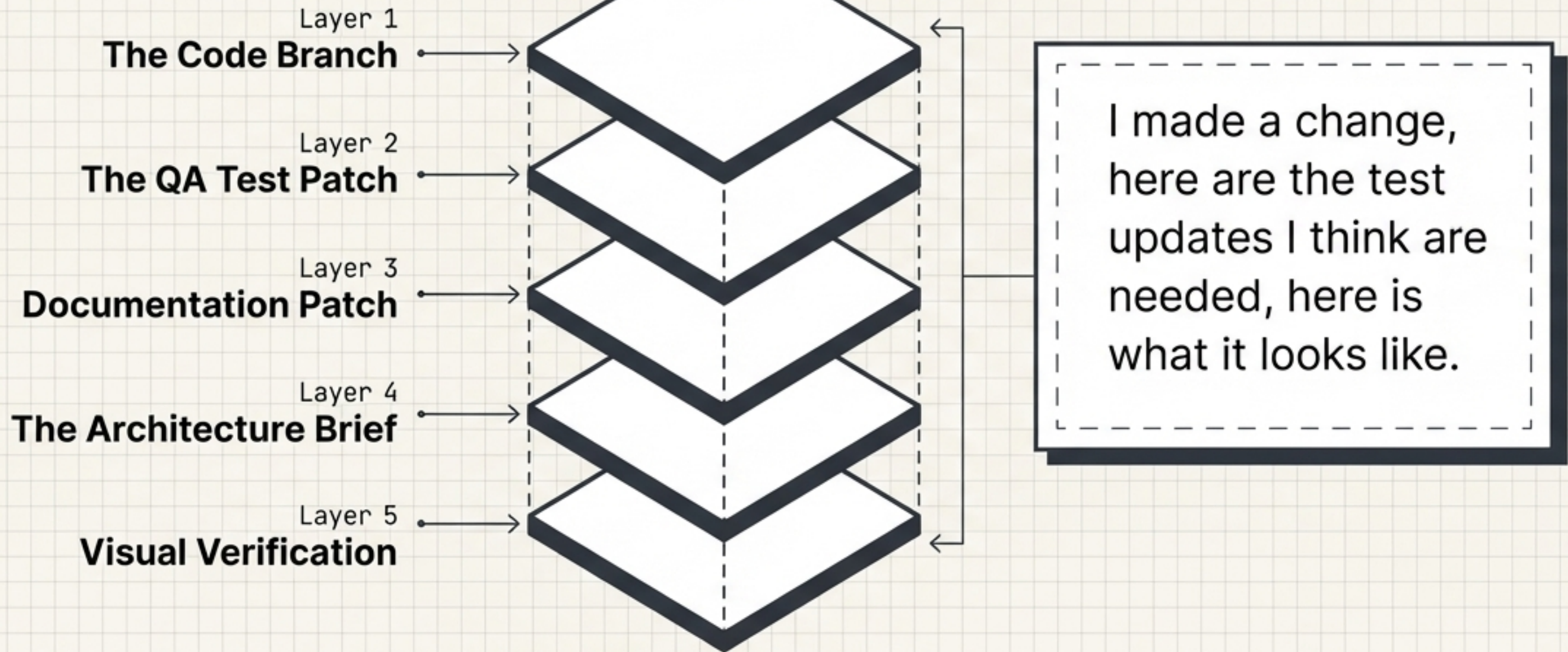
Clone QA

Pull a read-only clone of the QA repository.

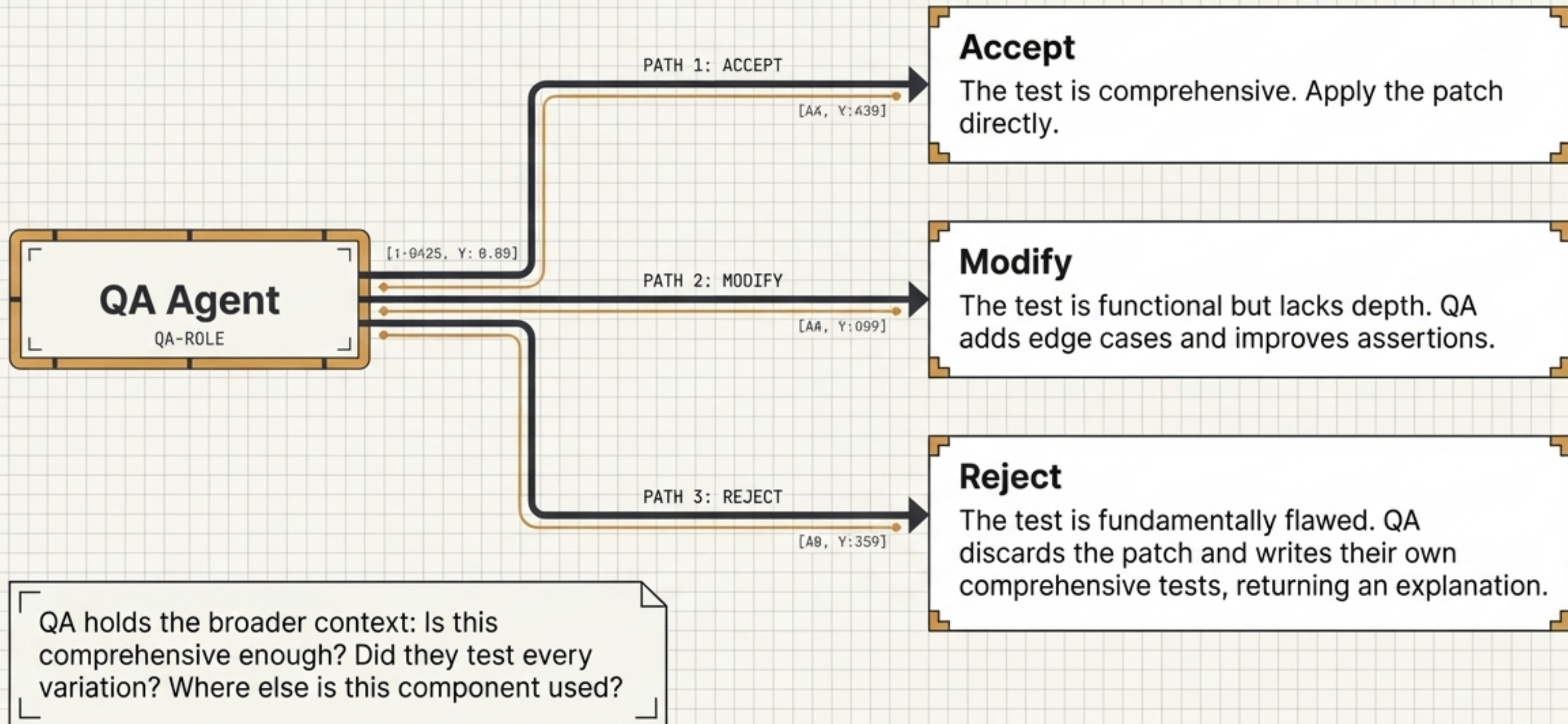
Update Tests

Alter broken tests to match new behaviour and verify they pass.

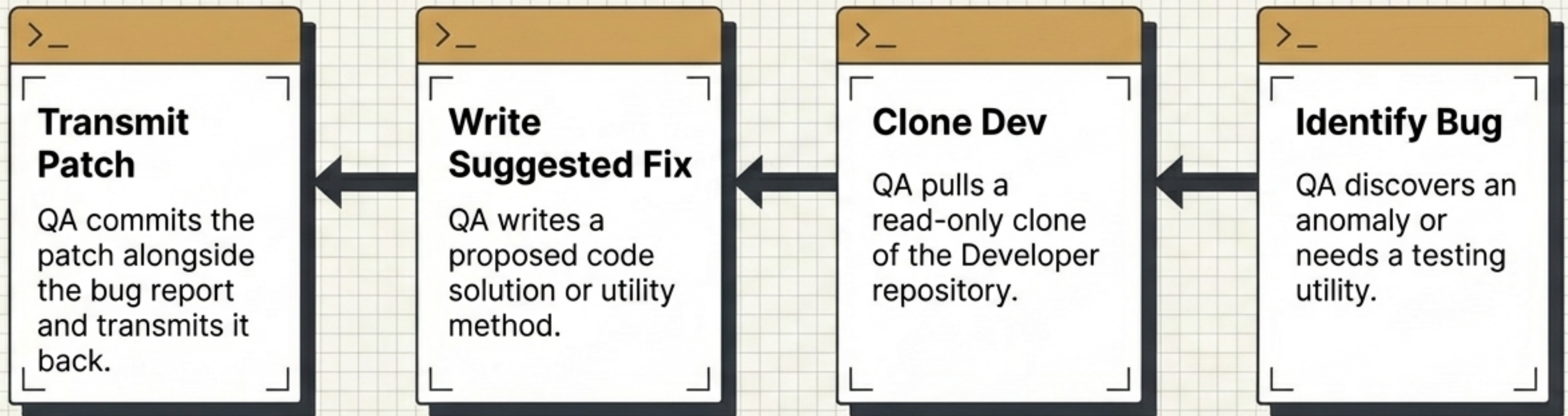
Communicating Exclusively with Code



QA Reviews and Decides



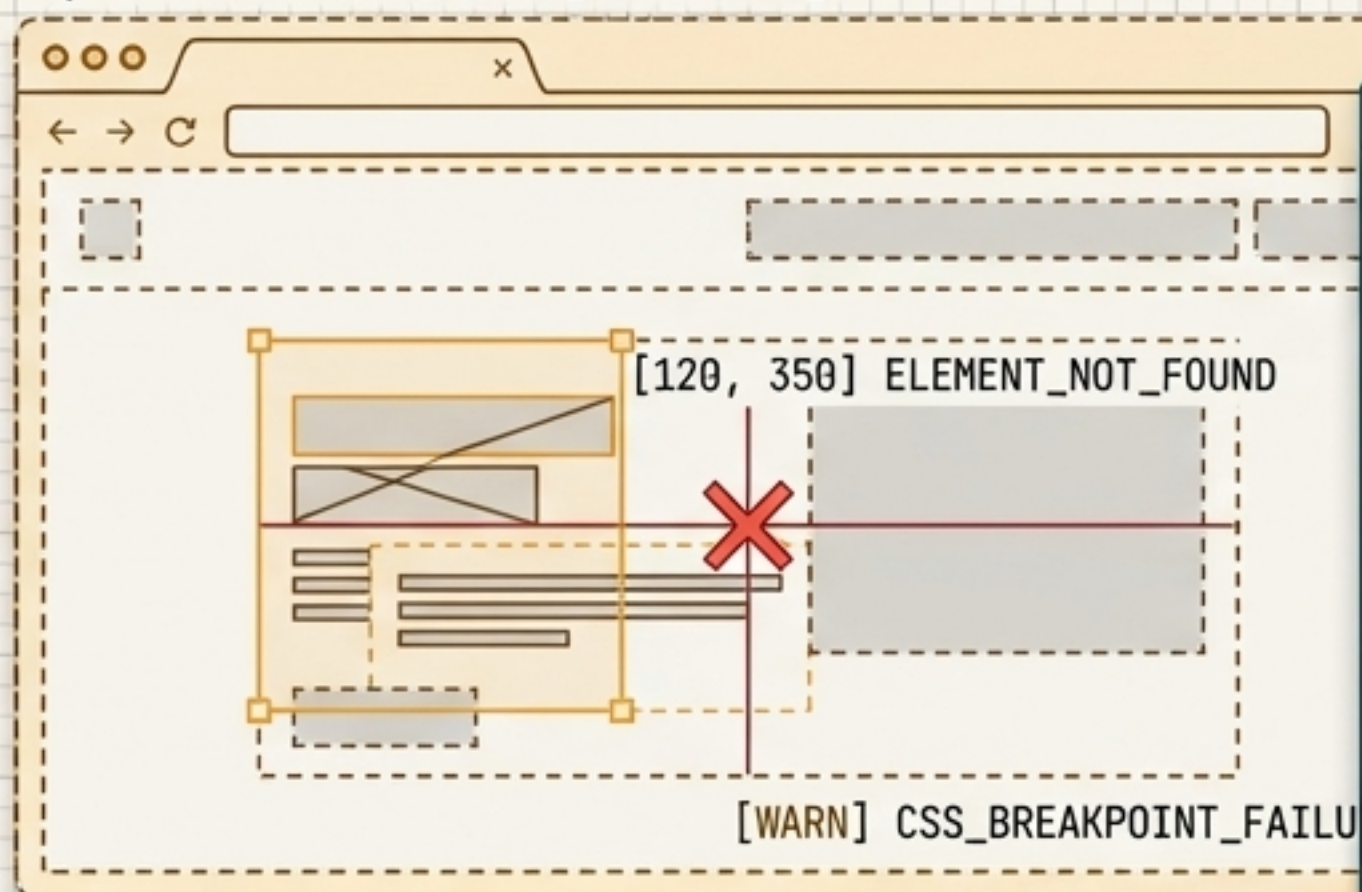
Bidirectional Code Recommendations



QA does not send ticket descriptions. They send readable, diffable code recommendations.

"Show, Don't Tell"

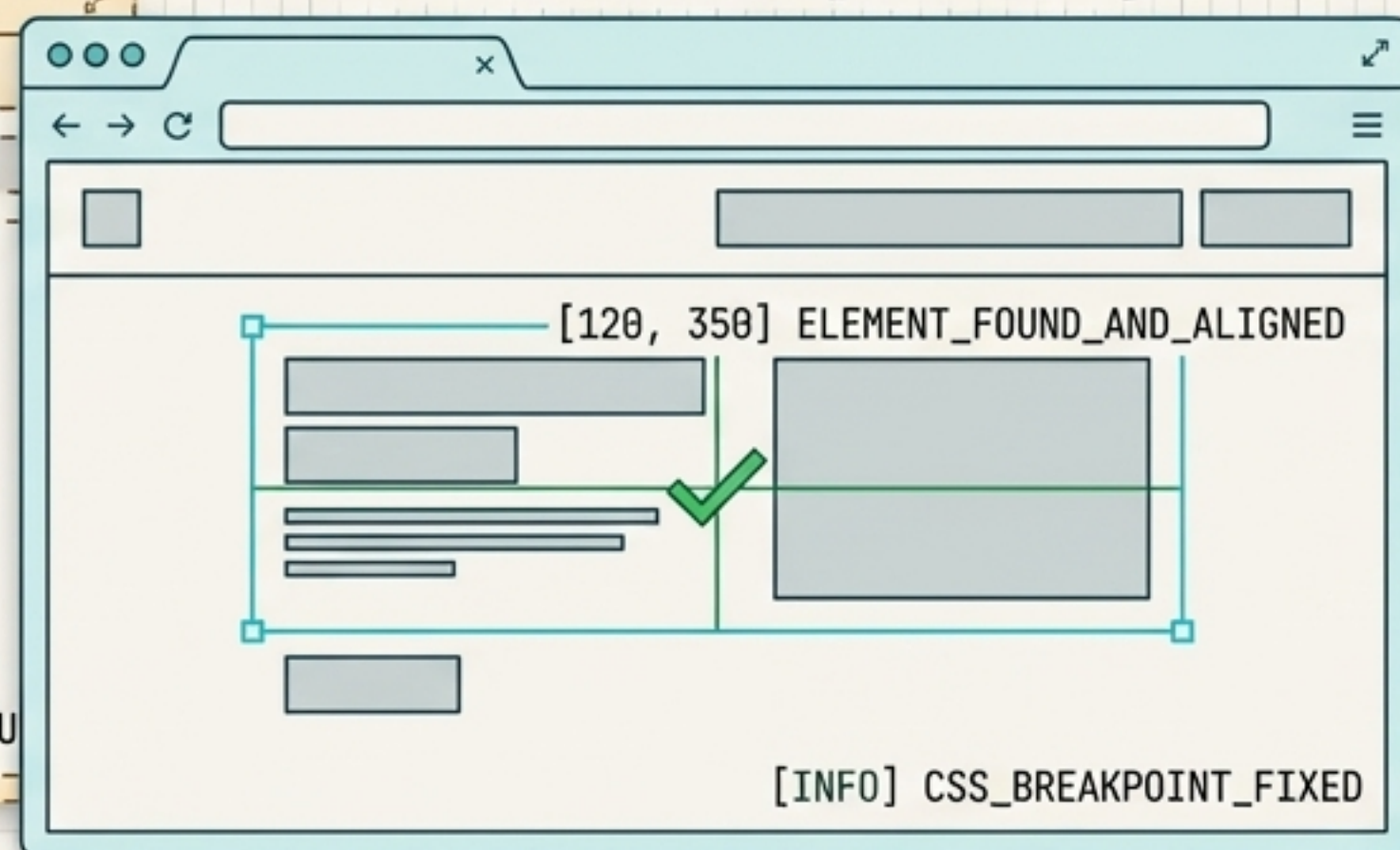
QA TEST OUTPUT [BROKEN]



QA sending a bug report:

This is what it looks like when it breaks.

DEV FIX OUTPUT [RESOLVED]

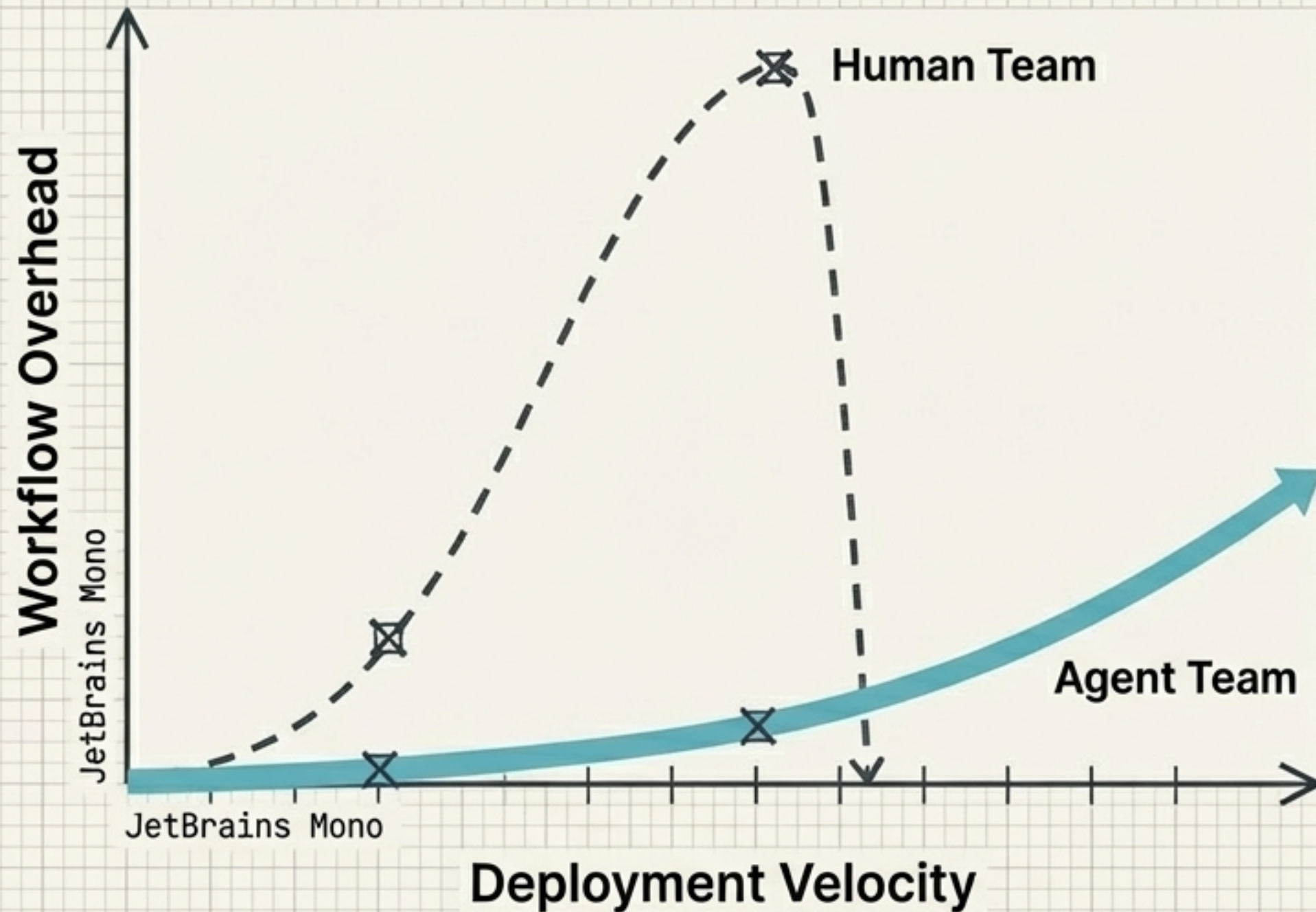


Developer sending the fix:

This is what it looks like after my change.

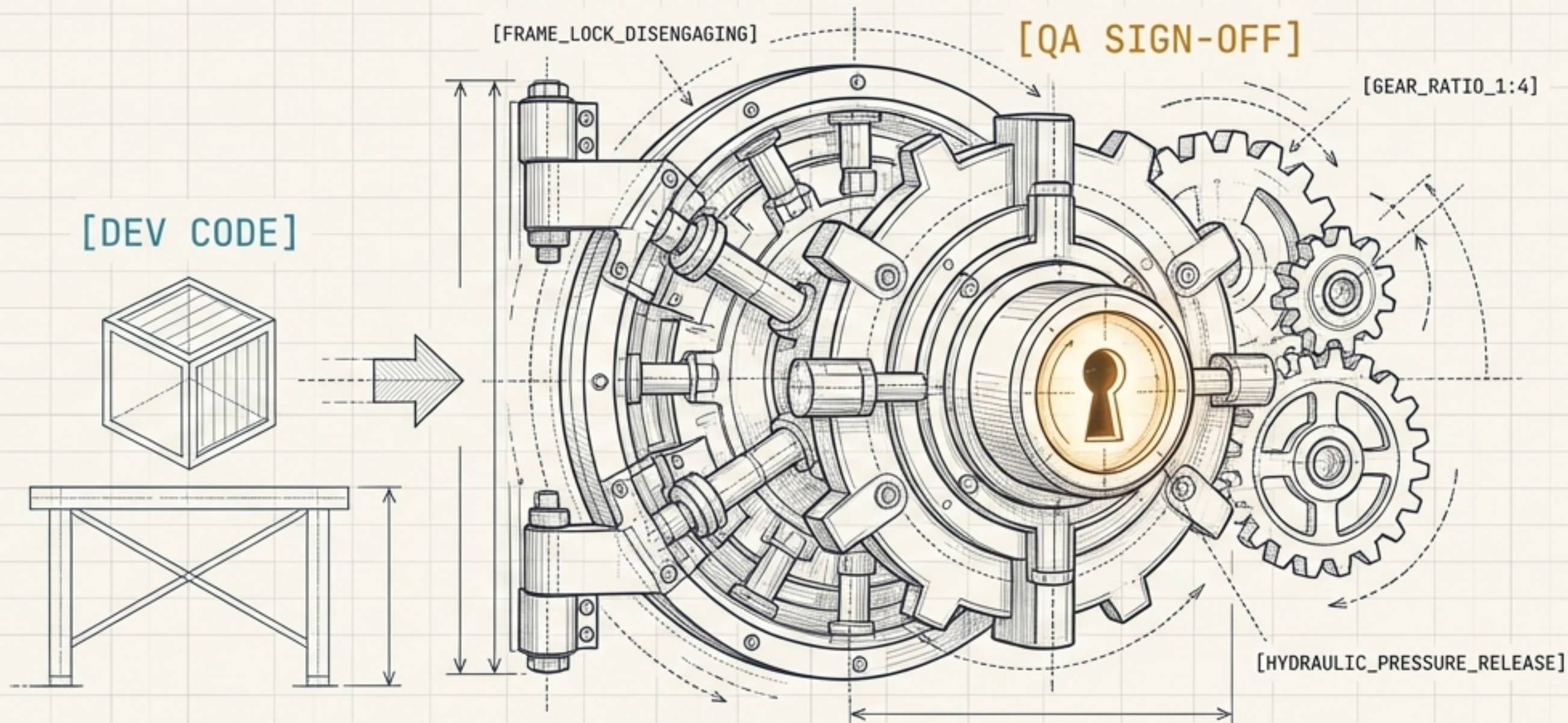
Driven by a unified Browser Automation API.
Both sides utilize identical, reproducible visual proof.

Zero-Overhead Collaboration



Agents communicate exclusively through precise, version-controlled artifacts (patches, briefs). Meetings and Slack messages are eliminated. Overhead approaches zero.

The Freedom to Break Things



QA Owns the Release Gate

A feature only goes to production when QA signs it off and all tests pass.

Developers Experiment Boldly

Liberated from the fear of breaking production, developers focus purely on solving the problem, knowing the QA safety net will catch anomalies.

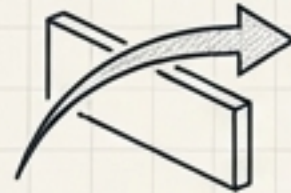
Resolving Decades of Friction

Then

QA frustrated by broken tests



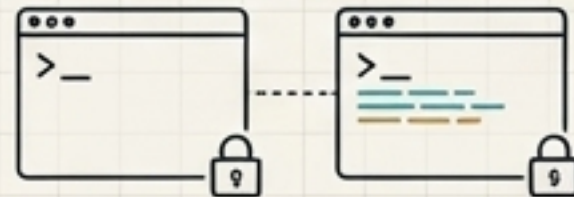
Changes thrown over the fence



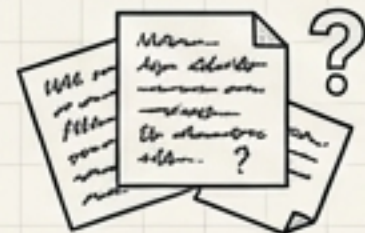
QA rewards brittle, un-breaking tests



Separate teams, isolated context



Undocumented, untraceable changes



Now

Broken tests expected; Devs update them



Patched securely across the Air Gap



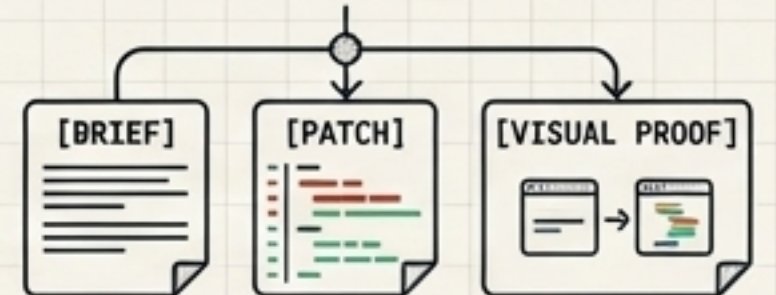
QA focuses on comprehensive coverage



Same agentic team, shared objectives



Briefs, patches, and visual proof for every commit



Architecture Acceptance Criteria

Infrastructure

- ✓ Test instructions live in READMEs
- ✓ Read-only clones utilized
- ✓ Strict Air Gap enforced (no cross-write access)

Execution

- ▣ Dev produces test patches alongside code
- ▣ Screenshots accompany code in both directions
- ▣ QA produces code patches for bugs
- ▣ QA signs off on release

Incidents

- ▣ Code change without test failure triggers coverage gap investigation
- ▣ Test failure without code change triggers infra investigation